5.1.3.4 Árboles de decisión

Son muy útiles para visualizar las diversas opciones de las que se dispone para resolver un problema o modelar un comportamiento. Se pueden convertir en reglas.

Se pueden aplicar métodos de inducción hacia atrás para descubrir el razonamiento que encierra la lógica del sistema.

Un **árbol de decisión** está compuesto de dos tipos de nodo, un **nodo condicional** y una **clase**. Los nodos condicionales son los nodos internos del árbol y la clase, los nodos hoja o terminales.

Cada condición (nodo condicional) es una pregunta para el sistema acerca de una variable y tiene dos posibilidades, que sea cierta o falsa. En función de eso seguirá un camino y otro. Los nodos finales son nodos hoja y nos dice la clase.

Fuente (https://es.wikipedia.org/wiki/Árbol_de_decisión#/media/Archivo:Arbol_decision.jpg)

Algunas de las implementaciones más famosas de estos algoritmos son el **ID3, J48 o C4.5 (clasificación) y M5 (regresión)**

Este algoritmo utiliza la **entropía** (medida de incertidumbre o de desorden) para ayudar a decidir qué atributo debe ser el siguiente en ser evaluado en el árbol. Es decir, el atributo seleccionado es aquel que deja la información más ordenada o, mejor clasificada.

$$Entropia(s) = \sum_{n=1}^{c} -p_i * Log_2 p_i$$

Donde c son los posibles valores de clasificación, S es el conjunto de todos los ejemplos y p_i es la proporción de ejemplos de S que están en la clase i

Y la ganancia:

$$Ganancia(S, A) = Entropia(S) - \sum_{v \in V(A)}^{|A|} \frac{|sv|}{|s|} Entropia(S_v)$$

Donde V(A) es el conjunto de todos los valores posibles para el atributo A y Sv es un subconjunto de S para el cual el atributo A tiene el valor v

Como selecciona el atributo más prometedor de entre todos, podemos concluir que realiza una **búsqueda voraz** entre los mejores atributos. Después aplica **Divide y vencerás** recursivamente con el problema.

Algoritmo

```
ID3(E,A,N):N
E: ejemplos, A: atributos, N: nodoRaíz
-----------------------------------
Si vacio(A) o Todos lso ejemplos de E pertenecen a una clase
  N.Clase = ClaseMayoritaria(E)
si no
  a = MejorAtributo(A) => aplicando entropía
  A = A - a => quitamos a
  Para cada valor v del atributo a de todas las instancias
    newNode = CrearNodo(a,v)
    N.AddHijos(newNodo)
    E = E - aquellos cuyo valor de de a sea v
    ID3(E,A,newNode)
return N
```

Ejemplo

Sitio de acceso A1	Cantidad gastada A2	Vivienda A3	Última compra A4	Clase
1	0	2	Libro	Bueno
1	0	1	Disco	Malo
1	2	0	Libro	Bueno
0	2	1	Libro	Bueno
1	1	1	Libro	Malo
2	2	1	Libro	Malo

Elegimos el mejor atributo:
$A1 = \frac{1}{6}.I_{10} + \frac{4}{6}.I_{11} + \frac{1}{6}.I_{12}$ Donde I_{1i} son los posibles valores del atributo 1 = {0,1,2}
Ahora calculamos I_{10}, I_{11} e I_{12} usando la Entropía:
I_{10} tenemos para la clase 0 (Bueno) y valor 0 un total de 1 ejemplos. Para la clase 1 (Malo) y valor 0 tenemos 0 ejemplos. El total de elementos con el valor 0 es 1, de ahi podemos deducir lo siguiente:

$I_{10} = -\frac{1}{1}log_2\frac{1}{1} - \frac{0}{1}log_2\frac{0}{1} = -1 * 0 + 0 = 0$ y para el resto...
$I_{11} = -\frac{2}{4}log_2\frac{2}{4} - \frac{2}{4}log_2\frac{2}{4} = 1$ (hay 2 buenos y 2 malos)
$I_{12} = 0$ porque hay 0 buenos y 1 malo

Aplicando la formula tenemos que:

$A1 = \frac{1}{6}.0 + \frac{4}{6}.1 + \frac{1}{6}.0 = 0,66$

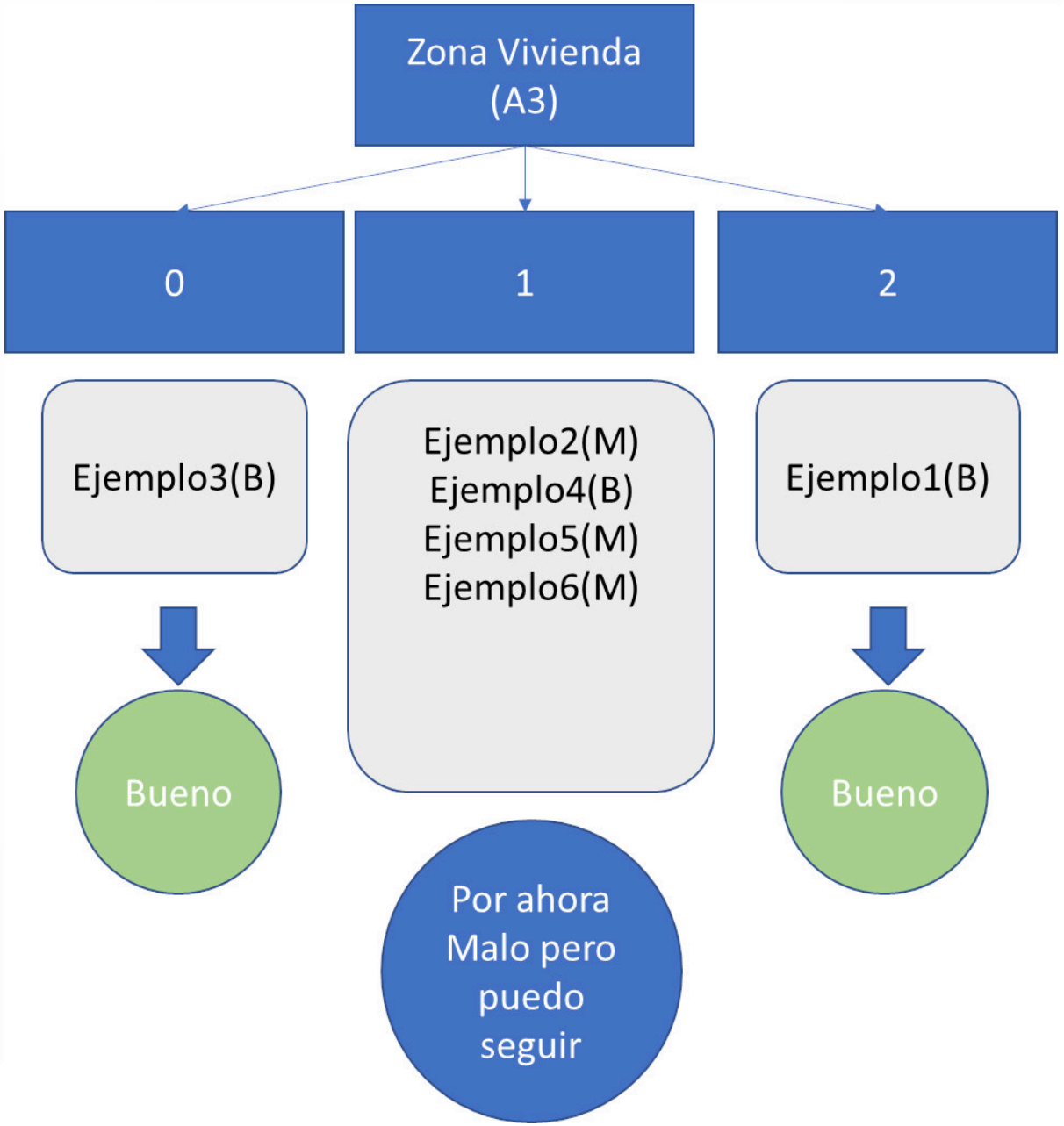
$A2 = 0,79$

$A3 = 0,54$

$A4 = 0,81$

El atributo seleccionado sería el de menor entropía (mayor orden) y por tanto seria el A3.

Esto ya nos deja el árbol con algunos terminales. Pero como puedo seguir podría seguir expandiendo y repitiendo la operación par el resto de atributos (A1,A2,A4)

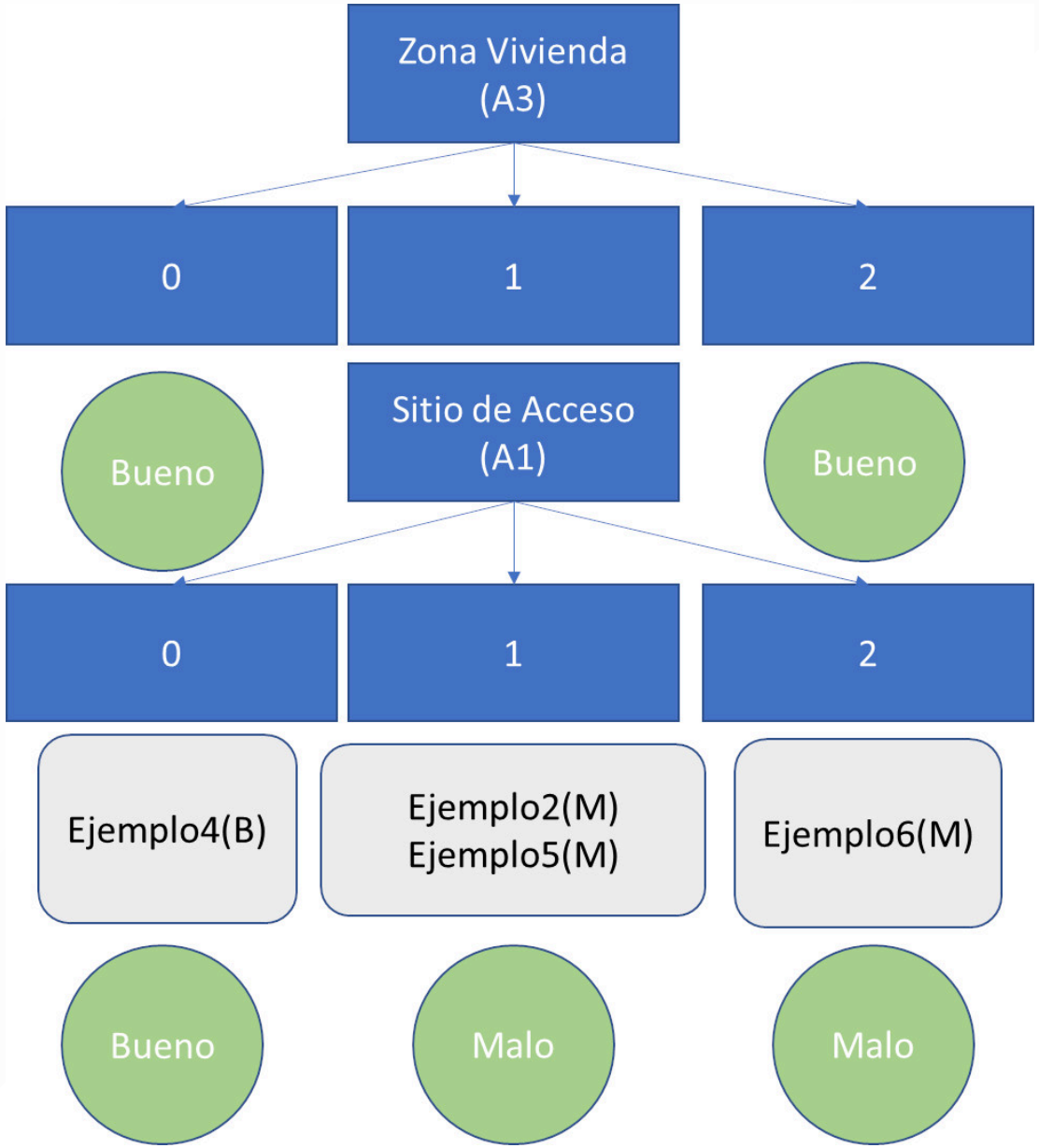

Zona Vivienda
(A3)
0
1
2
Ejemplo3(B)
Ejemplo2(M)
Ejemplo4(B)
Ejemplo5(M)
Ejemplo6(M)
Ejemplo1(B)
Bueno
Bueno
Por ahora
Malo pero
puedo
seguir

El siguiente atributo sería A1 sitio de acceso

$A1 = 0$
$A2 = 0,5$
$A4 = 0,23$

Que nos dejaría el árbol totalmente ordenado.

Nótese lo comentado antes, el DT hace una búsqueda voraz, jamas se replantea si habiendo puesto otro nodo inicialmente la configuración hubiera sido mejor (que en este caso no, pero en otros podría serlo.


Zona Vivienda
(A3)
0
1
2
Bueno
Sitio de Acceso
(A1)
Bueno
0
1
2
Ejemplo4(B)
Ejemplo2(M)
Ejemplo5(M)
Ejemplo6(M)
Bueno
Malo
Malo

Representación gráfica del algoritmo.

Los árboles de decisión dividen el espacio de soluciones mediante hiperplanos fijando una de las variables a un valor frontera.

Ventajas

- El entrenamiento es muy rápido
- Es fácil de interpretar los resultados por un humano, es un algoritmo de caja blanca.
- Para algunos problemas consigue una buena precisión.
- Se pueden convertir fácilmente en reglas.
- No requiere una preparación de los datos demasiado exigente.
- Puede trabajar con variables cualitativas y cuantitativas.

Desventajas

- Es muy dependiente al ruido de la entrada
- Los árbol de decisión tienden al sobre-entrenamiento
- No se puede garantizar que el árbol generado sea el óptimo
- Hay conceptos que no son fácilmente aprendibles pues los árboles de decisión ya que las particiones del espacio de soluciones que puede hacer son aquellas que son representables mediante una sucesión de hiperplanos. Si no hay una aproximación lineal al problema, puede que den un modelo poco efectivo.
- Se recomienda balancear el conjunto de datos antes de entrenar.

Ejemplo de DT en SKLearn

```
from sklearn.tree import DecisionTreeClassifier

from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import export_text
iris = load_iris()
decision_tree = DecisionTreeClassifier(random_state=0, max_depth=2)
X_train = iris.data[:-20]
y_train = iris.target[:-20]
X_test = iris.data[-20:]
y_test= iris.target[-20:]

decision_tree = decision_tree.fit(X_train, y_train)
r = export_text(decision_tree, feature_names=iris['feature_names'])
print(r)
decision_tree.score(X_test, y_test)
```

Ejemplo de DT en SKLearn

```
|--- petal width (cm) <= 0.80
|    |--- class: 0
|--- petal width (cm) >  0.80
|    |--- petal width (cm) <= 1.65
|    |    |--- class: 1
|    |--- petal width (cm) >  1.65
|    |    |--- class: 2
```

```
0.9
```